```
d = Demo.new

 # In all these cases, @attr is set to 99
d.attr = 99        # => 99
d.attr=(99)        # => 99
d.send(:attr=, 99) # => "return value"
d.attr             # => 99
```

## Element Reference Operator

*receiver*[ ‹*expr*›+ ]
*receiver*[ ‹*expr*›+ ] = *rvalue*

When used as an rvalue, element reference invokes the method [] in the receiver, passing as parameters the expressions between the brackets.

When used as an lvalue, element reference invokes the method []= in the receiver, passing as parameters the expressions between the brackets, followed by the *rvalue* being assigned.

# Aliasing

alias *new_name old_name*

This creates a new name that refers to an existing method, operator, global variable, or regular expression backreference ($&, $‘, $', and $+). Local variables, instance variables, class variables, and constants may not be aliased. The parameters to alias may be names or symbols.

```
class Integer
  alias plus +
end
1.plus(3)       # => 4

alias $prematch $`
"string" =~ /i/ # => 3
$prematch       # => "str"

alias :cmd :`
cmd "date"      # => "Sun Dec 25 12:30:40 CST 2022\n"
```

When a method is aliased, the new name refers to a copy of the original method’s body. If the original method is subsequently redefined, the aliased name will still invoke the original implementation.

```
def meth
  "original method"
end

alias original meth

def meth
"new and improved"
end

meth      # => "new and improved"

original  # => "original method"
```